

Nano-particle path: feeding a particle database into the adsorbates stage

design note (2026-07-09) + implementation status

implemented 2026-07-10

Contents

1 Status (as built, 2026-07-10)	1
2 Scope	3
3 Where the slab assumptions actually live	3
4 Suggestion 1: pipeline path (ingest, store, feed)	3
4.1 Ingest	3
4.2 Store into the standard tables	4
4.3 Workchain and orchestration changes	4
4.4 Flags for the implementer	4
5 Suggestion 2: adsorbate building process for particles	4
5.1 What exactly breaks, what is reusable	4
5.2 Route 1 (recommended): in-house placer on local outward normals	5
5.3 Route 2: adopt acat	5
5.4 Recommendation	5
6 Implementation split for pickup	6

1 Status (as built, 2026-07-10)

The path is IMPLEMENTED as a **fully separate chain** - one deliberate deviation from the original design below: particles are NOT materialized into DBStructure/DBSurface (Suggestion 1's "store into the standard tables"). Everything lives on DBNanoParticles; the slab pipeline and the particle pipeline share the chemistry code but no tables.

component	where	state
xyz import into DBNanoParticles	run_dir/import_nano_particles.py	DONE - box normalization (12 A vacuum), full formula + geometry hash in attributes, idempotent re-import; add_nano_particles fixed (wrote a non-existent composition column, now fills the sorted elements form)

component	where	state
adsorbates-on-particles chain	workchains/nano_particles.py (NanoParticleWorkChain)	DONE - consumes DBNanoParticles by elements (+ optional particles_range size filter), submits batches of 50 particles per CalcJob, computes CHE overpotentials with the SAME calculate_*_overpotential functions as the slab chain, stores per-site eta/dG/energy sets + best-set geometries into row.attributes["adsorbates"] ["<react relaxed bare particle back onto the row; idempotent per reaction/path; failed particles are marked Failed individually
particle adsorbates runner (Route 1)	codes/files/nano_particles.py	DONE - see "as built" notes in Suggestion 2 below
CalcJob + helpers	codes/nano_particles/{calculate_particles, parse_workchain}.py	DONE, parses the chain, adsorbates.py (as slab_adsorbates.py, the shared registry/validators) + _calculators.py + molecular references; submitted directly by class (no entry point needed), parsed with the registered generic sqs_parser
main.py wiring	workchains/main.py	DONE - _construct_particle_builder live (passes reaction/reaction_path, MLIP = settings.inputs['adsorbates']['model']), the nano status writes now target DBNanoParticles (were silent no-ops against DBComposition)

Open / consciously not done:

- **No materialization into DBStructure/DBSurface** (and therefore no automatic reuse of the DFT-verification chain or frontend surface queries for particles). The original plan is kept below in case that reuse is ever wanted; results would then need a migration from DBNanoParticles.attributes.
- The tilted-start sensitivity check (one ~30 deg off-normal starting orientation per site) from the physics flag below is not implemented – add it if a pathway looks anomalous on particles.
- Site-finder knobs (max_sites, default 24; surface_cn_max, default 9) are read from settings.inputs['adsorbates'] when present.
- Optional: aiida.calculations/aiida.parsers entry points for the nano CalcJob (only needed for factory access; direct class submission works).

2 Scope

We have a database of ready-made nano-particles and want to screen them through the existing AdsorbatesWorkChain (all seven reaction networks). There is **no generator mechanics** in this path: no systematic construction, no cook-and-relax exploration. The half-finished generator flow currently in `workchains/nano_particles.py` is not part of this design and can be stripped.

Two things need to change, and they are independent enough to be picked up separately:

1. the **pipeline path**: how particles get from their database into the rows the adsorbates stage consumes (section 2), and
2. the **adsorbate building process**: the placement step in `codes/files/adsorbates.py` is pymatgen-slab-specific and cannot place adsorbates on a cluster (section 3).

3 Where the slab assumptions actually live

Analysis result, so we do not re-derive it later:

- AdsorbatesWorkChain itself is geometry-agnostic. It pulls (`structure_uuid`, `surface_id`) pairs via `get_structure_uuid_surface_id` (a DBStructure x DBSurface join on composition), sorts by `slab['energy']`, ships `row.slab` to the CalcJob, and stores results with `add_surface_ml_adsorbate`. None of that cares about slab-ness.
- The reaction layer (`calculate_*_overpotential`), gas-phase references, ZPE handling, the bond-graph validator (`validate_relaxed_adsorbate`, “adsorbate = last n atoms”) and the MAD outlier filter operate on energies and graphs only. Reused untouched.
- The slab assumption is concentrated in ONE stretch of `codes/files/adsorbates.py`: `Slab.from_dict(data[0]) + get_adsorption_sites (pymatgen AdsorbateSiteFinder) + asf.add_adsorbate(..., reorient=True)`. ASF detects sites by z-height (a cluster would only get sites on its top cap), `reorient` rotates the molecule onto the global +z axis, and `selective_dynamics=True` would freeze the bottom half of a free particle in vacuum. Masquerading a particle as a Slab dict is therefore quietly wrong – the runner needs an honest particle branch.

4 Suggestion 1: pipeline path (ingest, store, feed)

4.1 Ingest

Strip NanoParticleWorkChain to an import chain: `load_particles -> relax -> store`. The `generate()` step and the `assert 1 == 2` scaffolding go away.

- **Source of particles**: the existing DBNanoParticles table stays the master record. Its loader `add_nano_particles(model, pairs, special_type)` exists (fixed 2026-07-10: it wrote a non-existent composition column, now fills elements in the sorted Au-Cu form that `main.py` queries by, and accepts per-row attributes). Importing an external particle file is one CLI call, DONE: `run_dir/import_nano_particles.py <file.xyz> - box normalization (re-center + 12 Å vacuum, pbc), full formula + geometry hash in attributes, idempotent re-import (duplicates skipped by hash)`. The import chain picks up rows with `status='Created'`.
- **The relax pass is mandatory even for pre-relaxed particles**. The stored particle energy becomes `slab_energy` in the adsorption energetics, so it must be computed with the SAME MLIP as the adsorbate relaxations (`settings.inputs['adsorbates']['model']`). Whatever energies the source database carries (DFT, another model) are not usable directly. The existing `_particle_relaxer` builder (`job_type='relax'`, batches of 10) does exactly this and is the one piece of the WIP chain worth keeping.
- **Box normalization**: re-center the particle and pad the cell (about 12 Å vacuum) during ingest rather than trusting whatever box the source database used. The site finder and the adsorbate need the headroom.

4.2 Store into the standard tables

NOT IMPLEMENTED - superseded (2026-07-10). The chain as built keeps particles entirely on DBNanoParticles (results in attributes, relaxed bare particle on the row); nothing below was materialized. Kept for the record in case DFT-verification / frontend reuse for particles is ever wanted – the arguments still hold.

Particles are materialized into the SAME tables every downstream consumer already reads (adsorbates, DBSurfaceMLAdsorbate, the DFT verification chain, frontend queries) – one code path, no parallel nano result tables:

- `add_structures(source='nano_particle', method=<adsorbates model>, ...)` for identity + energy (DBStructure / DBStructureVersion);
- `add_slab(uuid, None, {..., "energy": E, "miller_index": None, "kind": "nano_particle"})` for the feed row. `add_slab` derives the composition from the DBStructure row, so labels stay consistent;
- mark the DBNanoParticles row status='Stored' so re-ingest is idempotent.

Decision made here, revisit consciously if it hurts: particles are stored under their FULL formula (Cu₁₃, Cu₅₅Au₆), not the reduced one. `reduced_formula` collapses Cu₁₃ and Cu₅₅ into “Cu”; the lowest-10-by-energy selection in `run_adsorbs` would then compare total energies of different-size particles (meaningless) and silently drop sizes. With full formulas each size is its own composition: one AdsorbatesWorkChain per particle size, and the existing cap logic becomes harmless. Requires a small label override in `add_structures` (it currently hardcodes `reduced_formula`).

4.3 Workchain and orchestration changes

- AdsorbatesWorkChain: query and result storage unchanged. The cap of 10 in `run_adsorbs` is facet-competition logic; with full-formula compositions it mostly stops mattering (one row per particle). If many isomers per size are ever ingested, decide then whether isomers compete (keep lowest-E few) or all run.
- `codes/files/adsorbates.py`: branch on `data[0].get("kind") == "nano_particle"` – `Structure.from_dict` instead of `Slab.from_dict`, cluster site finder + placer instead of ASF (section 3). `repeat` fixed to (1,1,1). No `FixAtoms`: a free particle should be allowed to restructure under the adsorbate.
- `main.py`: the `should_run_nano_generator` branch stays but calls the import chain instead of the generator, gated on `DBNanoParticles.step_status` the same way other stages gate on `DBComposition.step_status`. After import the flow falls through to the normal adsorbates stage (the PD / synthesizability / SQS / surface-builder gates already return False on the nano path).

4.4 Flags for the implementer

- **Site symmetry explosion:** an icosahedral Cu₅₅ yields dozens of symmetry-equivalent ontop sites; without dedup every one is a wasted relaxation. Cheap mitigation first: fingerprint each site by its sorted neighbor-distance vector and drop duplicates. Proper point-group reduction only if that proves insufficient.
- The energies of clean particle, adsorbed particle and gas references all come from the same model by construction (ingest relax + adsorbates run) – do not shortcut the ingest relax.

5 Suggestion 2: adsorbate building process for particles

5.1 What exactly breaks, what is reusable

The adsorbate DEFINITIONS are fine: `_create_adsorbate_with_dummy` builds every intermediate as a pymatgen Molecule with the dummy X anchor at the binding site and the atoms at their bond offsets along internal +z. That convention is geometry-agnostic and is kept verbatim – it is the accumulated

house format across all seven reaction networks. Only the placement operator (`asf.add_adsorbate` with `reorient=True`) is slab-bound: it rotates the molecule onto the GLOBAL +z axis and trusts ASF's z-height site list.

5.2 Route 1 (recommended): in-house placer on local outward normals

IMPLEMENTED (2026-07-10) in `codes/files/nano_particles.py`. As-built notes on top of the sketch below: sites carry an outwardness guard ($\text{normal} \cdot \text{site-to-centroid} > 0$, drops spurious sites in concavities); duplicate sites collapse via a (element, distance)-environment fingerprint (an icosahedron's 62 raw sites reduce to exactly 1 ontop + 1 bridge + 1 hollow) and the remainder is capped by farthest-point sampling (`max_sites`, default 24, split across site types); azimuthal clash retries are 0/90/180/270 deg as sketched; the bare particle is relaxed in the same job (positions only, no `FixAtoms`) and provides the *"energy; gas references are relaxed once per 50-particle batch job. The registry, pathways and validator are imported from the slab runner (staged as `slab_adsorbates.py`), not copied. Validated end-to-end with EMT on Cu13 (HER complete set -> eta; OER OH correctly rejected by the identity check under EMT's broken chemistry).*

Roughly 60-80 lines of vector geometry, no new dependencies, reproduces the existing anchor convention exactly (so slab and particle numbers stay comparable):

- **Surface detection:** coordination number from covalent radii (the bond cutoff machinery already exists in the runner); surface atom = CN below threshold.
- **Per-atom outward normal:** $n_i = \text{normalize}(r_i - \text{mean}(r_{\text{neighbors}(i)}))$. The coordination shell sits on the material side, so this points outward automatically and stays correct for non-convex or elongated particles (unlike a radial-from-centroid direction).
- **Sites:**
 - ontop: position r_i , normal n_i ;
 - bridge (bonded surface pair i,j): midpoint, $\text{normalize}(n_i + n_j)$;
 - hollow (surface triangle i,j,k): centroid, $\text{normalize}(n_i + n_j + n_k)$.
- **Placement:** rotation R mapping $[0,0,1]$ onto the site normal (Rodrigues, handle the antiparallel edge case), then $\text{coords}' = R @ \text{molecule_coords} + \text{site_position}$, append, drop the X dummy. Because the intermediates carry their site-to-atom offsets in internal coordinates, this is exactly what ASF's `reorient=True + distance=0 + translate=False` combination does for slabs.
- **Clash handling:** particle sites near edges/vertices have neighbors above the local tangent plane (slabs mostly do not). After placement run the existing `has_reasonable_distances`; on failure retry the same site with the molecule rotated azimuthally about the normal (0/90/180/270 degrees) before discarding. The slab path just breaks on a bad distance; the retries salvage usable sites.

Everything downstream (validator, MAD filter, gas references, energy sets) is untouched: the "adsorbate = last n atoms" convention is preserved.

5.3 Route 2: adopt acat

The ACAT package does this professionally: `ClusterAdsorptionSites` enumerates typed, symmetry-deduplicated sites on nanoparticles (ontop/bridge/fcc/hcp/4fold with proper local normals), and `acat.build.add_adsorbate_to_site` handles oriented placement including bidentate modes. It would solve the symmetry-explosion problem for free.

Costs: a new dependency in every MLIP venv on the HPC side (the runner is a staged file executed in the code environment), and the X-dummy intermediate registry would need conversion to acat's adsorbate spec – real friction across seven reaction networks.

5.4 Recommendation

Start with Route 1. It slots into the runner as one alternative branch, keeps the staged runners dependency-free, and reuses the adsorbate registry and validator with zero changes. Handle sym-

metry duplicates with the fingerprint dedup. If the site taxonomy later proves limiting (subsurface sites, bidentate intermediates, per-facet statistics), evaluate `acat` then – it would replace only the same two functions (site finder + placer), so nothing in Route 1 locks us out.

Physics flag to decide up front: on a curved surface, upright placement along the local normal is a starting guess that matters more than on flat slabs – tilted binding modes on edges/vertices are common. The MLIP relaxation finds them from an upright start most of the time; if a pathway looks anomalous on particles, add one tilted starting orientation (about 30 degrees off-normal) per site as the cheap sensitivity check.

6 Implementation split for pickup

How it was actually cut (2026-07-10) – all three pieces landed, but not along the lines sketched here:

- (a) ingest: `run_dir/import_nano_particles.py` + fixed `add_nano_particles` – DONE. The full-formula/add_slab materialization was dropped with the standard-tables storage (see Status).
- (b) cluster site finder + placer: its own runner `codes/files/nano_particles.py` importing the shared chemistry from the slab runner, NOT a branch inside `codes/files/adsorbates.py` – DONE, validated on Cu₁₃ (EMT, HER/OER).
- (c) `main.py`: `_construct_particle_builder` wired (reaction/reaction_path added, model from settings) – DONE.